

Alumni Connect Project Report

1. Executive Summary

Alumni Connect is a role-based full-stack web platform designed to connect students, alumni, and administrators in one career-oriented ecosystem. The project combines mentoring, job discovery, community interaction, profile management, administrative control, and AI-assisted productivity tools inside a single application.

From the codebase, the project is built with a React frontend and a Flask backend, using MongoDB as the main datastore. Authentication is handled through JWT tokens stored in HttpOnly cookies, which gives the application a session-based feel while still using token-based authorization under the hood. The system also includes Socket.IO for realtime communication and a Gemini-backed optional AI layer for selected features.

In practical terms, the platform is trying to solve a real campus problem: students often need direct access to alumni guidance, verified mentors, role-relevant opportunities, and resume support, while administrators need visibility, control, and moderation tools to keep the ecosystem useful and safe.

The strongest implemented parts of the project are:

- role-based login and protected access
- alumni verification by admin
- mentorship recommendation and request flow
- realtime conversation system
- internal and LinkedIn-oriented job workflows
- AI-assisted resume generation and content help
- forum sentiment tracking and admin oversight

Some parts of the platform are still closer to prototype or demo mode, especially a few dashboards and listing screens that currently rely on hardcoded or mock data rather than full backend integration.

2. High-Level Architecture

The project is split into two primary applications:

- ``frontend/``: React + Vite client application
- ``backend/``: Flask API and realtime backend

Frontend stack

The frontend uses:

- React
- React Router
- Tailwind CSS
- Framer Motion
- Recharts

- Socket.IO client
- jsPDF and HTML/PDF export helpers

The frontend is responsible for:

- route protection
- role-aware navigation
- form handling
- dashboard rendering
- chat UI
- job portal UI
- resume builder UI
- admin interface
- calling backend APIs

Backend stack

The backend uses:

- Flask
- Flask-CORS
- Flask-JWT-Extended
- Flask-SocketIO
- PyMongo
- bcrypt
- python-dotenv

The backend is responsible for:

- authentication and cookie-based session handling
- role validation
- MongoDB CRUD operations
- profile updates
- mentorship recommendation and request creation
- message and conversation APIs
- forum posting and sentiment scoring
- job posting and applying
- LinkedIn organization OAuth and publishing
- AI enhancement endpoints
- admin metrics and moderation

Database model

MongoDB is the main database. Based on the controllers and queries, the main collections are:

- `users`
- `conversations`
- `messages`

- ``job_posts``
- ``job_applications``
- ``forum_posts``
- ``platform_settings``

This means the application is designed in a document-oriented style rather than a relational schema style. Profile information is nested inside the user document, which simplifies user-centric reads and updates.

3. Backend Structure and Responsibility

The backend follows a fairly understandable separation of concerns.

``src/app.py``

This is the application factory. It:

- loads environment variables
- configures CORS for the frontend
- configures JWT cookies
- disables JWT CSRF protection in the current setup
- initializes MongoDB
- initializes Socket.IO
- registers all blueprints

This file acts as the main composition layer of the backend.

``src/config/database.py``

This module initializes MongoDB and stores a reusable ``mongo`` extension object. The backend expects a Mongo URI and database name from environment variables, with local defaults already provided.

``src/routes/*``

Each feature domain gets its own route file. These routes are mostly thin wrappers around controller functions. The current route groups are:

- auth
- profile
- admin
- chat
- mentorship
- forum
- jobs
- LinkedIn
- AI tools

``src/controllers/*``

This is where most business logic lives. The controllers perform:

- request parsing
- validation
- database reads/writes
- role checks
- response shaping

`src/services/\*`

These service modules handle reusable logic such as:

- mentor matching
- sentiment analysis
- Gemini integration
- Socket.IO event emission

This is a good direction architecturally because it keeps controllers from becoming too bloated.

4. Authentication and Access Control Flow

Authentication is one of the most important implemented sections of the project.

Registration flow

During registration:

1. the backend receives email, password, and role
2. it checks whether the user already exists
3. password is hashed with bcrypt
4. user data is formatted through `UserSchema.format\_user`
5. the document is inserted into MongoDB

Role-specific behavior exists during registration:

- students are immediately marked verified
- admins are immediately marked verified
- alumni are created with `is\_verified = False`

This means alumni onboarding is intentionally controlled by admin approval.

Login flow

During login:

1. backend validates email, password, and requested role
2. it fetches the user by email
3. bcrypt password comparison is performed
4. requested role must match the user's actual role
5. alumni login is blocked if not yet admin-approved

6. last login timestamp is updated
7. a JWT access token is created
8. the token is written into an HttpOnly cookie

This is a clean role-aware login flow. It prevents users from logging in through the wrong portal and prevents unapproved alumni from entering the system early.

Session restoration

On frontend load, ``AuthContext`` calls ``/api/auth/me``. Since the cookie is automatically sent by the browser, the backend can identify the user and return their role/name/email. This lets the frontend restore the logged-in session without storing tokens in local storage.

Route protection

The frontend uses ``ProtectedRoute`` and the backend uses JWT checks plus role validation. This gives you two layers:

- frontend UX-level route blocking
- backend actual permission enforcement

That is the right model because frontend checks alone are never enough.

5. User Model and Profile System

The user model is defined in ``backend/src/models/user_model.py``.

Core fields

Each user has:

- email
- hashed password
- name
- role
- verification state
- created date
- last login
- account status

Nested profile

The nested ``profile`` object holds:

- PRN or ID
- graduation year
- headline
- bio
- skills

- career goals
- mentorship topics
- availability
- location
- communication preference
- company
- social links

This structure is suitable for a mentorship and professional networking platform because it centralizes the signals needed for matching and profile viewing.

Profile update flow

The backend profile update controller uses MongoDB ``$set`` on nested ``profile.*`` fields. That is a useful design choice because it avoids replacing the whole profile object and reduces the chance of accidental data loss during partial updates.

Profile usage across the app

Profile data feeds multiple features:

- mentorship matching
- public profile visibility
- alumni profile editor
- edit-profile form
- job scoring
- resume generation defaults

That makes the profile system one of the central pillars of the platform.

6. Role-Based Product Experience

The platform is clearly designed around three personas.

Student experience

Students can:

- register and log in
- edit professional profile
- view dashboard
- get AI-ranked mentor matches
- request alumni mentors
- open realtime mentorship conversations
- browse and apply for jobs
- browse LinkedIn-marked jobs
- use resume builder
- post in forum
- use article writer

- use campus assistant

From a workflow perspective, the student journey is the most complete end-user journey in the project.

Alumni experience

Alumni can:

- register and wait for approval
- log in after admin verification
- maintain a profile
- participate in conversations with students
- post jobs
- publish jobs to organization LinkedIn
- access student-facing network tools

The alumni workflow focuses on contribution: mentorship, job sharing, and profile visibility.

Admin experience

Admins can:

- log in through a dedicated portal
- approve pending alumni
- list all students
- list verified alumni
- review overview metrics
- inspect conversations
- change moderation status of conversations
- access LinkedIn connection and publishing workflow

The admin role acts as trust manager, moderator, and platform operator.

7. Mentorship Matching Module

The mentorship feature is one of the most important and interesting sections of the project.

Recommendation source

The backend recommendation controller:

- loads the current student
- loads all verified alumni except the student's own email
- computes ranked mentor recommendations
- optionally asks Gemini to improve scores and match reasons

Matching logic

The local fallback scoring considers:

- overlapping student skills and alumni skills
- semantic overlap from profile/headline/bio/tokens
- alumni company presence
- alumni availability
- overlap between student career goals and alumni mentorship topics
- optional area of interest filter from the request

This is not just a random listing. It is a rule-based recommendation layer that gives reasonably explainable matching behavior.

Gemini enhancement

If Gemini is enabled:

- student payload is sent
- mentor payload list is sent
- Gemini returns ranked scores and short match reasons
- local results are updated and resorted

This creates a hybrid AI design:

- deterministic local fallback
- smarter ranking when external AI is available

That is a solid product decision because the feature still works even without the external model.

Mentorship request flow

When a student requests a mentor:

1. the backend validates the mentor ID
2. finds the student and alumni records
3. builds sorted participant email list
4. checks if a conversation already exists
5. creates a mentorship conversation if not
6. inserts a system message announcing the mentorship request

This flow ties mentor discovery directly into the chat system, which makes the experience feel connected rather than fragmented.

8. Realtime Conversation and Communication System

The chat system is implemented with Flask-SocketIO and a React conversation UI.

Conversation model

Conversations include:

- type
- status

- moderation status
- participant emails
- participant metadata
- mentorship request metadata
- last message
- timestamps

Messages include:

- conversation ID
- sender email
- sender name
- sender role
- text
- kind
- created time

Chat flow

The main communication flow is:

1. user opens conversations page
2. frontend fetches conversation list
3. frontend fetches selected conversation messages
4. socket connection is established using JWT cookie identity
5. user joins conversation room
6. sending a message inserts it into MongoDB
7. backend updates conversation metadata
8. backend emits realtime events to relevant rooms
9. frontend updates live

Moderation

The backend blocks message sending if a conversation is:

- muted
- closed

This is important because it means moderation is not cosmetic. Admin status changes actually affect behavior.

Frontend experience

`ConversationHub.jsx` provides:

- conversation sidebar
- message history
- live updates
- role-aware back navigation

- message send handling

This is one of the stronger integrated frontend/backend features in the codebase.

9. Job Portal and Application Workflow

The job system is more than a simple listing board.

Job creation

Eligible posters:

- alumni
- admin
- college role if such users exist

Job documents contain:

- title
- company
- description
- tags
- application link
- location
- poster info
- source
- LinkedIn publishing fields
- visibility
- creation time

Job listing

When jobs are listed for a user:

- user profile is loaded
- jobs are fetched
- previous applications are checked
- each job is enriched with:
 - match score
 - applied status
 - application count

Job scoring

The match score considers:

- overlap between user skills and job tags
- career-goal/tag relevance

This makes the student job board feel somewhat personalized rather than static.

Job application flow

Students can:

1. open external application link
2. confirm whether they applied
3. backend stores application record
4. job application count is updated
5. realtime socket event is emitted

This is a practical compromise when the actual application occurs outside the platform.

LinkedIn board

The frontend exposes:

- standard jobs
- LinkedIn jobs

The LinkedIn board is basically a filtered job view for posts marked or published as LinkedIn-oriented opportunities.

10. LinkedIn Integration

This is a notable advanced feature in your project.

Purpose

The LinkedIn module lets the organization connect a LinkedIn account and publish jobs from the platform backend.

Status and connection flow

Admin can:

1. request LinkedIn connect
2. be redirected to LinkedIn OAuth
3. return to callback
4. backend exchanges code for token
5. backend stores organization auth data in `platform\_settings`

Stored information includes:

- access token
- scope
- organization URN
- connected by
- connected at
- token expiry

Publishing flow

When a user publishes a job to LinkedIn:

1. backend verifies permissions
2. verifies LinkedIn config
3. verifies organization token exists
4. loads job
5. builds post commentary
6. sends LinkedIn post request
7. stores resulting LinkedIn URL and publish metadata
8. emits realtime job update

This is a strong feature because it extends the platform beyond internal campus use into outward job promotion.

Product value

This gives admins and alumni a way to:

- create jobs once
- display them inside the platform
- optionally amplify them through LinkedIn

That is a meaningful integration for a career-oriented product.

11. Forum and Sentiment Monitoring

The forum is a lighter community module, but it includes moderation-aware logic.

Forum posting

Authenticated users can create posts with:

- text
- user role
- user identity
- company if present
- sentiment metadata

Sentiment analysis

The current sentiment model is rule-based. It counts simple positive and negative keyword hits and produces:

- numeric score
- label
- flagged boolean
- hit counters

This is not a full NLP model, but it is enough to support a basic moderation signal and demo health metric.

Community health

Forum list response also includes a derived community health object:

- average score
- label such as Healthy, Monitor, or At Risk

This is useful for dashboards and gives the forum a measurable moderation layer.

Strength and limitation

Strength:

- forum is actually integrated with backend persistence and moderation metrics

Limitation:

- replies and likes are still basic and not fully implemented as interactive social features

12. AI Tools Module

The AI tools section is broad and one of the most product-rich parts of the app.

Resume generator

The resume generator:

- accepts structured form data
- improves summary, skills, education, experiences, certifications, and projects
- falls back to local text transformation logic
- optionally uses Gemini for stronger polishing
- returns generated sections and an overall score
- supports multiple output templates

This module is more than a toy endpoint. It includes a lot of transformation logic and is backed by a large frontend resume builder workflow.

Resume section improver

This endpoint lets the user improve one section at a time. It supports repeated attempts so users can compare alternate phrasings. That is a thoughtful UX choice.

Article writer

The article generator is simpler. It creates structured article drafts using topic and audience. It currently works more like a templated content assistant than a fully dynamic long-form AI system.

Campus assistant

The assistant endpoint is lightweight and intent-based. It provides canned guidance depending on whether the user asks about:

- resume
- jobs
- mentorship
- general help

This makes it a guided navigation assistant rather than a true conversational agent.

13. Frontend Application Structure

The frontend is route-driven and role-aware.

Main application structure

`App.jsx` defines public and protected routes for:

- public pages
- student portal
- alumni portal
- admin portal

Auth context

`AuthContext.jsx` is responsible for:

- restoring session
- storing current user in memory
- redirecting after login
- logout handling

This keeps authentication state centralized and reduces duplication.

Shared components

Important shared UI pieces include:

- `ProtectedRoute`
- `RoleSidebar`
- `ConversationHub`

These components help unify behavior across roles.

Service layer

Frontend service modules wrap backend APIs:

- `jobService.js`
- `mentorshipService.js`
- `forumService.js`

- `chatService.js`
- `aiToolsService.js`
- `api.js`

This is a good separation because page components stay cleaner and HTTP behavior stays centralized.

14. Screen-by-Screen Product Coverage

Student dashboard

The student dashboard presents the student experience as a career readiness hub. It highlights mentors, jobs, conversations, and resume tools. However, some displayed values are still hardcoded demo values rather than live backend analytics.

Mentorship matching screen

This is one of the better-finished screens. It uses real backend data, supports area-of-interest filtering, and lets the student directly start the mentor request flow.

Conversation hub

This screen is strongly integrated. It fetches conversations and messages, joins socket rooms, and updates live.

Job portal

This is a central cross-role screen. It supports:

- student browsing and applying
- alumni/admin posting
- LinkedIn publish actions
- realtime updates

Resume builder

This is one of the most feature-heavy frontend modules. It supports:

- multiple steps
- structured input
- section-by-section AI improvement
- template preview
- PDF export for resumes

Forum

The forum screen is connected to backend data and shows sentiment health indicators directly in the UI.

Article writer and assistant

These are functional utility screens for extra value-added support.

Alumni dashboard

The alumni dashboard is visually polished but currently relies on static chart data. It looks more like a presentation layer placeholder waiting for live metrics.

Student profiles page

This page currently uses mock student data, so it is not yet fully connected to the backend.

Forgot/reset password

These screens are currently UI-only and do not connect to a backend reset token flow.

15. Admin and Governance Capabilities

The admin module has a clear governance role.

Alumni approval

Pending alumni accounts can be listed and approved manually. This gives the platform a trust gate before alumni can access privileged features.

Overview metrics

The admin overview endpoint aggregates counts for:

- students
- verified alumni
- pending alumni
- conversations
- flagged chats
- mentorship requests
- messages
- job posts
- applications
- forum posts
- flagged forum posts

This gives the administrator a compact operational snapshot.

Conversation moderation

Admin can inspect conversations and apply moderation states:

- active
- flagged
- muted
- closed

This is one of the most operationally meaningful backend features because it directly affects chat behavior.

16. Current Strengths

The codebase shows several clear strengths.

Strong full-stack integration

Several features are truly end-to-end:

- mentor request to chat flow
- job create/list/apply flow
- admin approval flow
- auth/session restoration flow

Clear feature modularity

The backend and frontend are both organized by domain features, which makes the project easier to understand and extend.

Good role-based product thinking

The app is not just technically role-protected. Each role has different workflows and different UI navigation, which shows product design intent.

Realtime value

Socket.IO is used in meaningful places instead of being included as a decorative technology choice.

AI fallback design

The presence of local fallback logic in mentorship and resume tools is especially valuable because the app remains usable even without Gemini configuration.

17. Current Gaps and Technical Debt

The project also has some important limitations.

Static or mock sections

These areas appear incomplete or demo-oriented:

- alumni dashboard charts
- student dashboard stat summaries
- student profiles page
- forgot password flow
- reset password flow

Testing gap

There is no real app-level automated test suite visible in the repo. That increases risk when changing:

- auth logic
- job flow
- chat behavior
- profile shape
- moderation behavior

Dependency drift

The backend requirements still include packages such as SQLAlchemy and psycpg2 while the current backend architecture is clearly MongoDB-first. This may indicate leftover dependency history.

Hardcoded environment assumptions

Some values are hardcoded around local development, including:

- frontend origin
- backend API origin
- localhost ports

This is common during development, but deployment will need more environment-based configuration.

Inconsistent old code remnants

A few components appear to be older or unused variants. These are not necessarily harmful, but they increase maintenance noise.

18. Suggested Next Development Priorities

If the goal is to mature this project into a stronger portfolio or production-ready platform, the best next steps would be:

Priority 1: complete unfinished flows

- implement forgot password backend flow
- implement reset token validation and password update
- replace mock student profile listing with live backend data
- connect alumni dashboard to real metrics

Priority 2: improve data integrity and validation

- add request validation schemas
- normalize field naming consistently across frontend and backend
- enforce stronger validation for profile and job payloads

Priority 3: add testing

- auth controller tests
- profile update tests
- mentorship recommendation tests
- job application tests
- admin moderation tests

Priority 4: production hardening

- move secrets and origins fully to environment variables
- revisit JWT CSRF protection settings
- add logging and error monitoring
- handle token expiry and refresh strategy explicitly

Priority 5: enrich platform intelligence

- stronger sentiment moderation
- conversation audit trail
- recommendation feedback loops
- richer admin analytics

19. Final Assessment

Alumni Connect is a strong applied full-stack project with a clear problem statement and a meaningful multi-role product design. It goes beyond a standard CRUD demo by including:

- role-sensitive workflows
- admin trust controls
- realtime communication
- AI-enhanced recommendation and writing support
- LinkedIn integration
- moderation visibility

The project already demonstrates solid practical engineering ability, especially in how multiple modules are connected into a coherent product story. Its biggest opportunity now is not inventing new features, but finishing the partially mocked areas, tightening validation, and adding automated testing.

Overall, this is best described as a campus career-networking and mentoring platform with AI assistance, moderation, and external job publishing capability. The foundation is real, the core workflows are credible, and with a few focused improvements it can become both a strong academic project and a compelling portfolio application.